APIVizer

v4.0.0 PHANTOM

User Guide

Autonomous API Security Exploitation Agent

May 2026

Table of Contents

1. Introduction

APIVizer is an autonomous API security penetration testing tool that analyzes Postman collections to identify vulnerabilities, generate real exploit proofs, and produce comprehensive security reports. It requires **zero external dependencies** — only Python 3.8+ standard libraries.

Key Capabilities:

- Autonomous vulnerability discovery from Postman collections
- Real exploit execution with proof-of-concept generation
- OWASP API Security Top 10 (2023) coverage
- AI-powered analysis via MCP (Model Context Protocol) integration
- HTML reports with CVSS scoring and video PoC demonstrations
- Zero dependencies — runs with Python standard library only

2. Prerequisites

Required:

- Python 3.8 or higher
- A Postman Collection file (.postman_collection.json)
- Network access to the target API endpoints

Optional (for AI-powered mode):

- VS Code or JetBrains IDE with GitHub Copilot
- MCP server configuration (see Section 6)

Verify Python:

```
python --version
```

Expected: Python 3.8+ (e.g., Python 3.11.5)

3. Quick Start (5-Minute Setup)

Step 1: Place files in the same folder:

```
apivizer.py
YourAPI.postman_collection.json
```

Step 2: Open terminal in that folder.

Step 3: Run APIVizer:

```
python apivizer.py
```

Step 4: APIVizer auto-detects Postman collections in the directory and begins scanning.

That's it! Reports are generated automatically in the same folder.

4. Running a Scan

APIVizer automatically discovers all **.postman_collection.json** files in its working directory. Simply run the script and it handles the rest.

Basic Usage:

```
python apivizer.py
```

With Postman Environment (for variables/auth tokens):

Place your **.postman_environment.json** file in the same directory. APIVizer will auto-detect and use environment variables for authentication tokens, base URLs, and other configured values.

What happens during a scan:

1. Collection parsing — Extracts all endpoints, methods, headers, and bodies
2. Environment loading — Resolves {{variables}} from environment files
3. Vulnerability analysis — Tests each endpoint against 40+ attack patterns
4. Exploit execution — Fires real requests to verify vulnerabilities
5. Report generation — Creates HTML report with findings and PoCs
6. Video PoC creation — Generates animated HTML proof-of-concepts

5. Understanding the Output

After a scan completes, APIVizer generates the following files:

File	Description
*_exploit_report.html	Basic exploit report with all findings
*_exploit_report_enhanced.html	Full report with executive summary, CVSS scores, charts
*_pocs/ folder	Individual video PoC HTML files for each finding
*_pocs/INDEX.html	Index page linking to all video PoCs
*_pocs/ALL_POCS.json	Machine-readable JSON of all findings

6. MCP Integration (AI-Powered Mode)

APIVizer can integrate with GitHub Copilot via the Model Context Protocol (MCP) for AI-enhanced analysis. This enables natural language interaction and smarter exploit generation.

Setup (VS Code):

1. Install GitHub Copilot extension in VS Code
2. Open Settings → search 'MCP'
3. Add MCP server configuration pointing to apivizer.py
4. Restart VS Code — APIVizer auto-connects to MCP

MCP Configuration (settings.json):

```
{ "mcp.servers": { "apivizer": { "command": "python", "args":
["path/to/apivizer.py"] } } }
```

Note: MCP mode is optional. APIVizer works fully standalone without it. See APIVizer_MCP_Setup_Guide.pdf for detailed MCP setup instructions.

7. Scan Modes & Options

Standalone Mode (Default):

Run directly from terminal. APIVizer uses its built-in AI engine for analysis.

```
python apivizer.py
```

MCP Mode (AI-Enhanced):

When connected via MCP, you can interact with APIVizer through Copilot Chat. The tool auto-detects MCP connectivity and switches mode automatically.

Attack Categories Tested:

- Authentication Bypass — Tests for missing/weak auth on all endpoints
- Rate Limiting — Detects absence of rate limiting on sensitive operations
- Security Headers — Checks for 15+ security headers (HSTS, CSP, X-Frame, etc.)
- HTTP Method Override — Tests X-HTTP-Method-Override bypass techniques
- Token Security — Verifies token handling (URL params, predictability, expiry)
- Cookie Security — Checks Secure, HttpOnly, SameSite flags
- Server Disclosure — Detects version information leakage
- Sensitive Data Caching — Verifies Cache-Control headers on sensitive endpoints
- Injection Attacks — SQL injection, XSS via parameter manipulation

8. Reading the Exploit Report

Open the ***_exploit_report_enhanced.html** file in any browser. The report contains:

Report Sections:

- Cover Page — Target, date, risk score, methodology
- Executive Summary — High-level findings count and risk assessment
- Findings Table — All vulnerabilities sorted by CVSS severity
- Detailed Findings — Each vulnerability with description, impact, evidence, and remediation
- Risk Matrix — Visual CVSS distribution chart
- Methodology — OWASP API Top 10 mapping

Severity Levels:

Severity	CVSS Range	Action Required
CRITICAL	9.0 - 10.0	Immediate fix required
HIGH	7.0 - 8.9	Fix within 24-48 hours
MEDIUM	4.0 - 6.9	Fix within 1-2 weeks
LOW	0.1 - 3.9	Fix in next sprint

9. Video PoC Files

APIVizer generates animated HTML-based video PoCs that simulate a terminal session showing the exploit being executed. These are useful for demonstrating findings to stakeholders.

How to use:

1. Open the ***_pocs/INDEX.html** file in a browser
2. Click any finding to watch the animated PoC
3. Each PoC shows the exact request/response that proves the vulnerability

Sharing PoCs:

PoC files are standalone HTML — no server needed. Share the entire *_pocs/ folder or individual HTML files via email, Teams, or any file share.

10. Troubleshooting

Issue	Solution
No collections found	Ensure .postman_collection.json files are in the same directory as apivizer.py
Connection timeouts	Verify network access to target API. Check VPN if needed.
Empty report	Check that environment file has correct base URL and auth tokens
MCP not connecting	Restart IDE. Verify MCP config in settings.json. See MCP Setup Guide.
Python version error	Upgrade to Python 3.8+. Run: python --version to verify.
Permission denied	Run terminal as administrator or check file write permissions.

11. Best Practices

• **Always use environment files** — They provide auth tokens and base URLs for accurate scanning.

• **Run from target network** — Ensure you have network access to the API endpoints being tested.

• **Update collections regularly** — Export fresh collections from Postman before each assessment.

• **Review reports with context** — Some findings may be acceptable risks depending on architecture.

• **Share PoCs responsibly** — Reports contain sensitive security data. Follow your org's data handling policy.

• **Run scans in SIT/UAT** — Avoid running against production unless explicitly authorized.

• **Keep apivizer.py updated** — Newer versions have additional attack patterns and bug fixes.

— End of User Guide —